

Desenvolvimento de Serviços Web e Testes com Java

DR3 — Assessment

Aluno: Guilherme de Moraes China Costa

Matrícula: 44671579804

Branch: atividade-guilherme_china

Repositório: github.com/guilhermecosta/api-vendas-AT

Relatório de evidências dos exercícios 1 a 12

Exercício 1 — produtos-service no Eureka

Abra `http://localhost:8761` com Eureka e produtos-service ativos. Capture a janela inteira, mantendo a URL visível e a linha **PRODUTOS-SERVICE** legível em *Instances currently registered with Eureka*.



Figura 1 — Registro do produtos-service no Eureka.

Exercício 2 — Branch e Pull Request

No GitHub, abra a aba **Pull requests** e entre no PR da branch `atividade-guilherme_china` para `main`. Capture o cabeçalho mostrando o estado **Open**, as duas branches e o título do PR.



Figura 2 — Pull Request da atividade.

Exercício 3 — Inicialização na porta 8084

Execute `docker compose logs --tail=100 fornecedores-service`. Capture as linhas `Tomcat started on port 8084` e `Started FornecedoresServiceApplication`, incluindo o comando no terminal.



Figura 3 — Inicialização do novo microserviço.

Exercício 4 — Cinco fornecedores no H2

Acesse `http://localhost:8084/h2-console`. Use JDBC URL `jdbc:h2:mem:fornecedoresdb`, usuário `sa` e senha vazia. Execute `SELECT * FROM FORNECEDOR;` e capture a consulta e exatamente os cinco registros iniciais.



Figura 4 — Dados carregados automaticamente no H2.

Exercício 5 — Endpoints GET e status 404

No Postman ou Insomnia, capture `GET http://localhost:8084/fornecedores` com status 200 e os registros. Depois capture `GET http://localhost:8084/fornecedores/999` com o status **404 Not Found** visível.



Exercício 6 — fornecedores-service no Eureka

Atualize `http://localhost:8761` . Capture a janela inteira com a URL e a linha **FORNECEDORES-SERVICE** marcada como **UP**.



Figura 6 — Registro do fornecedores-service no Eureka.

Exercício 7 — Configuração centralizada

Primeiro abra `http://localhost:8888/fornecedores-service/default` e capture a resposta contendo `server.port: 8084`, H2 e Eureka. Depois execute `docker compose logs --tail=100 fornecedores-service` e capture `Located environment` junto de `Tomcat started on port 8084`.



Exercício 8 — Acesso pelo Gateway

Abra `http://localhost:8085/fornecedores-service/fornecedores` . Capture a janela inteira, deixando a porta **8085** visível na URL e a lista de fornecedores na resposta.



Figura 8 — Roteamento dinâmico pelo Spring Cloud Gateway.

Exercício 9 — Cadastro com POST

No Postman ou Insomnia, envie `POST http://localhost:8084/fornecedores` com JSON `{"nome": "Zeta Componentes", "cnpj": "67.890.123/0001-45"}`. Capture requisição, status **201 Created** e o `id` devolvido. Se esse CNPJ já foi usado, reinicie o `fornecedores-service` ou altere apenas o CNPJ.



Figura 9 — Novo fornecedor persistido.

Exercício 10 — Consulta ao produtos-service com Feign

Com Eureka, produtos-service e fornecedores-service ativos, abra

`http://localhost:8084/fornecedores/produtos` . Capture a URL e a lista de produtos devolvida pelo fornecedores-service.



Figura 10 — Comunicação entre microsserviços via OpenFeign e Eureka.

Exercício 11 — Docker Compose e Gateway

Execute `docker compose up -d --build` e, quando concluir, `docker compose ps`. Capture todos os containers com status **Up**. Em seguida capture novamente a resposta em `http://localhost:8085/fornecedores-service/fornecedores`.



Exercício 12 — GitHub Actions

Depois do push, abra a aba **Actions** do repositório, selecione o workflow **Compilar fornecedores-service** e abra a execução mais recente. Capture o nome do workflow, o commit/branch e o job **build** concluído com check verde.



Figura 12 — Compilação Maven automatizada com Java 17.